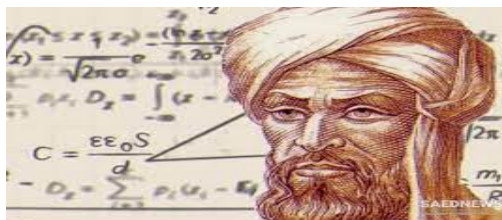


Chapitre 2

PROGRAMMATION PYTHON



1 Introduction

Les algorithmes sont partout, au cœur de nos ordinateurs, de nos smartphones, des objets connectés qui envahissent chaque jour davantage notre société. Mais, qui a inventé l'algorithme ? et c'est quoi un algorithme ? Le mot "algorithme" vient du nom du mathématicien **Abu Djafar Muhammad ibn Musa, dit al-Kharezmi** né en 780. qui est à l'origine de l'algèbre. Son traité intitulé **Hisab aljabr w'al-muqabala** est considéré comme le premier manuel d'algèbre. Il y traite entre autres de la résolution des équations du premier et du second degré.

2 Définitions et notions de base

2.1 Définition

Un algorithme est une succession d'instructions élémentaires, faciles à faire et non ambiguës, déterminées de façon unique par les données initiales, et fournissant la réponse à un problème posé. C'est une suite finie d'instructions, écrites en langage naturel, qui peuvent être exécutées les unes à la suite des autres pour résoudre un problème.

Les algorithmes peuvent être représentés de plusieurs façons :

- Des étapes écrites
- Des flèches de direction (map)
- Des organigrammes
- ...

La conception d'un algorithme passe par des étapes fondamentales :

1. Spécification du problème à résoudre
2. Préparation des moyens de résolution (Données, contraintes,)
3. Résolution étape par étape du problème
4. Affichage des résultats si nécessaire



Exemple d'algorithme

La résolution de $f(x) = ax^2 + bx + c$ dans $\mathcal{R}$ avec a non nul.

1. Spécification du problème : Résolution d'une équation de second degré ;
2. calcul de Δ
3. Si $\Delta > 0$ alors : $x_1 \leftarrow \frac{-b - \sqrt{\Delta}}{2a}$, $x_2 \leftarrow \frac{-b + \sqrt{\Delta}}{2a}$

4. sinon si $\Delta = 0$, $x \leftarrow \frac{-b}{2a}$
5. Sinon pas de solutions dans $\mathcal{R}$
6. Affichage des résultats

2.2 Langage de programmation

Un programme est la traduction d'un algorithme du langage naturel en un langage reconnu par la machine. Ces langages sont appelés [langages de programmation](#)

Les langages de programmation sont dits de bas niveau s'ils sont proches du langage machine comme le langage de programmation **Assembleur**. Ces langages sont difficiles à écrire par rapport aux langages de haut niveau comme **Python, Java ...**

Il existe deux sortes de langages de programmation :

- langage compilé (C, C++, ...)
- langage interprété (Python, HTML, ...)

La différence entre les deux c'est que dans le langage compilé, le compilateur traduit une bonne fois pour toute un code source en un fichier indépendant exécutable. Par contre le langage interprété doit être interprété à chaque lancement du programme, pour traduire au fur et à mesure le code source en code machine.

Nom du langage	Apparu en
Assembleur	1950
Basic	1964
Pascal	1970
C	1973
C++	1983
Python	1991
Java	1994

2.3 Le Langage Python

- Langage interprété, originellement écrit en C
- Open-source, portable et disponible sur Unix, Windows, MacOS, etc.
- Syntaxe claire et simple
- Orienté objet Types nombreux et puissants
- Large spectre d'applications



Le langage de programmation Python est un langage puissant, que l'on peut trouver partout. Parmi les champs d'utilisation de Python :

- Jeux vidéo 2D ;
- Calcul scientifique
- traitement d'image, de son, de vidéo ...
- Systèmes d'intelligence Artificielle (IA)
- Traitement de données massives (Big Data)
- Des applications Web (Framework Django)
- ...

Sous windows, Python doit être installé en le téléchargeant depuis le site officiel. Pour certaines versions de Linux (Debian, Ubuntu ...), Python se trouve déjà installé.

Plusieurs IDE sont présents pour programmer en Python :

- Spyder ;
- PyCharm ;
- Pyzo
- Jupyter
- ...

3 Variables, Données, Opération

3.1 Les variables et L'affectation

Les instructions d'un algorithme mettent en jeu des données (numériques, texte, etc) qui peuvent être saisies par l'utilisateur, résulter d'un calcul de l'ordinateur, etc.

Il est pratique d'enregistrer ces données dans la mémoire de l'ordinateur pour pouvoir les réutiliser au fur et à mesure du programme.

Les espaces mémoire où ces données sont conservées sont appelés [variables](#).

Comme son nom indique, la variable peut changer de contenu durant le programme via l'opération [d'affectation](#).

**Exemple d'affectation en algorithmique**

$x \leftarrow \frac{-b - \sqrt{(\Delta)}}{2a}$ est une affectation
 $x \leftarrow 10$

En Python, on remplace le symbole d'affectation algorithmique $\leftarrow$ par un simple `=`

**Exemple en python**

`x = (-b - sqrt(Δ))/(2 * a)` est une affectation en python
`x = 10` On affecte à `x` la valeur 10
`x = a` On affecte à `x` le contenu de la variable `a`

**Attention !**

`x = a` n'est pas commutative. l'affectation se fait dans un seul sens et cette instruction se lit : on affecte à `x` la valeur de `a`.

3.2 Nommage des variables

En partie par obligation liée au langage, et en partie par **[convention]** pour l'enseignement, un nom de variable doit :

- Commencer par une lettre **[minuscule]**
- ne contenir que des lettres sans accents, des chiffres et le tiret de soulignement `_` (appelé tiret "underscore" ou encore "tiret du 8")
- Ne contenir pas des espaces
- utiliser des noms de variables **[significatis]**
- n'utiliser pas des mots clés réservés

**Exemple**

`note = 15.5`
`Annee = 2021`
`date_naissance = "03/05/2002"`

Mots réservés en Python

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

3.3 Exercices

**Exercice**

Quelles seront les valeurs de A et de B à la fin de ce programme

```
1 A=7
2 B=5
3 A=A+B
4 B=A+B
5 B=B-10
```



Exercice

Remplir le tableau par les valeurs des variables pour chaque ligne exécutée du programme :

```
1 A=7
2 B=3
3 C=2
4 B=B-3
5 A=A*B
6 A=B+A
7 B=C
8 C=C
```

Ligne	A	B	C
1			
2			
3			
4			
5			
6			
7			
8			

3.4 Types de données

Chaque donnée et chaque variable possède un certain type. Il est important pour l'ordinateur de savoir de quel type est une variable pour lui réserver sa mémoire nécessaire.

Nom	nom python	en anglais	Exemple
Entier	int	Integer	13
Réel	float	float	13.5
chaîne de caractères	str	string	"bonjour"
complexe	complex	complex	3+4j
booléen	bool	boolean	True, False

pour savoir le type d'une variable sous python on utilise la fonction type

```
In [5]: A=44
In [6]: type(A)
Out[6]: int
In [7]:
```

3.5 Les opérations de base

Action	Entrée	Sortie
Addition	3 + 10	en sortie 13
Soustraction	17 - 10	en sortie 7
Multiplication	10 *10	en sortie 100
Division	15 / 2	en sortie 7.5
Division entière	15 // 2	en sortie 7
Modulo	15 % 2	en sortie 1
Puissance	4 ** 2	en sortie 16
Affichage	print(" bonjour")	bonjour
Affichage	print(bonjour)	Affiche le contenu de la variable bonjour

3.6 Les fonctions d'entrées/Sorties

La fonction `print()`

 La syntaxe de la fonction `print()` est la suivante:

```
1 print(valeur(s), sep=' ', end = '\n' )
```

- **Valeur(s)** : une chaîne de caractères, une variable, un nombre, une opération, ...
- **sep** : (Facultatif) Le séparateur qui sépare les éléments à afficher s'il y en a beaucoup.
- **end** : (Facultatif) Spécifiez ce qu'il faut afficher à la fin. Par défaut c'est un retour à la ligne "`\n`"

 Exemple:

```
1 print('B', 'N', 'N', sep='A', end='\n') #En sortie BANAN
```

Pour afficher des chaînes formatées, on peut utiliser aussi la méthode `.format()` :

 Exemple en Python:

```
1 x=10
2 y=20
3 print("x={} et y {}".format(x,y))      #En sortie x=10 et y=20
4 print("x={1} et y={0}".format(x,y))    #En sortie x=20 et y=10
5 prenom=input() #Lit et affecte la valeur saisie par l'utilisateur
```

Les accolades sont réservées pour classer les arguments de la méthode `.format()`. Par défaut l'ordre d'apparition des arguments si on ne met rien entre les accolades.

La fonction `input()`

Pour insérer des données via le clavier, on utilise la méthode `input()` :



```
1 prenom=input() #Lit et affecte la valeur saisie par l'utilisateur
2 print(prenom) #Affiche la valeur lue
```

On peut écrire un message à afficher à l'utilisateur



```
1 prenom=input("Écrire ton prénom") #Affiche le message puis lit la valeur
2 print(prenom) #Affiche la valeur lue
```

Par défaut la valeur lue au clavier est une chaîne de caractères, la conversion en un autre type de données s'appelle [transtypage ou cast en Eng](#).

Pour la convertir en un entier ou un réel, on utilise la fonction `int()` et `float()` :



```
1 n=int(input("Donner un nombre entier")) #Affiche le message puis lit la valeur
2 print(n) #Affiche l'entier lu
3 x=eval(input("Donner une expression mathématique")) #Affiche le message puis lit
4 print(x) #Affiche l'expression évaluée
```

la fonction `eval()` évalue une expression mathématique et affecte le résultat si c'est l'expression est bien évaluée.

3.7 Les opérateurs de comparaison

Python comprend différents opérateurs permettant d'effectuer des opérations entre les opérandes, c'est-à-dire entre des variables, des littéraux ou encore des expressions.

Opérateur	En Python	Signification
=	==	égal à
<	<	inférieur stricte-ment
>	>	supérieur stricte-ment
≤	<=	inférieur ou égal
≥	>=	supérieur ou égal
≠	<> ou !=	différent de
∈	in	Dans
∉	not in	n'est pas dans

Exemple:

```
1 x=0
2 if x in {0,1,2,3}:
3     print(x,"existe")
4 else:
5     print(x, "n'existe pas")
```

4 Instructions

4.1 Instruction composée

Indentation

En python, la structuration des blocs d'instructions composées se fait grâce à un décalage par des espaces appelé **indentation**.

les lignes consécutives qui ont la même indentation appartiennent au même bloc.

Dans d'autres langages de programmation on utilise souvent les **accolades ouvrantes { et fermantes }** pour désigner un **bloc**

Conditionnelle

Exemple:

```
1 if Condition:
2     Bloc d Instructions 1
3 else:
4     Bloc d Instructions 2
```

Se traduit en :

```
1 if moyen >= 10 :
2     print("Tu as réussi")
3 else:
4     print("Tu n'as pas réussi")
```

Ou encore:

```
1
2 if Condition 1 :
3     Bloc d Instructions 1
4 elif Condition 2 :
5     Bloc d Instructions 2
6 else :
7     Bloc d Instructions 3
8 Suite du programme
```

Exemple

```
1 if moyen >=18:
2     print("Tres bien")
3 elif moyen >=16:
4     print("T.Bien")
5 elif moyen >=14:
6     print("Bien")
7 elif moyen >= 12:
8     print("A.Bien")
9 elif moyen>=10:
10    print("Passable")
11 else:
12    print("Insuffisant")
```

Répétitive

boucle *for*

Les instructions de boucles permettent en général de répéter une instruction ou un bloc d'instructions plusieurs fois. Sa syntaxe est la suivante :

La syntaxe générale de la boucle *for* est la suivante :

```

1 for i in range(borne_inf, borne_sup, pas):
2     instructions
    
```

Avec

- *i* : est une variable quelconque ;
- borne_inf : elle est facultative, si on ne le met pas, la boucle commencera de 0.
- borne_sup : elle est obligatoire, c'est la valeur maximale que *i* va attendre.
- pas : il est facultatif, il représente la pas avec lequel la variable *i* change de valeur, elle peut être soit un entier positif soit négatif. Par défaut le pas=1

Exemple en algorithmique

```

1 pour i de 0 à 10 faire:
2     Ecrire("Bonjour")
    
```

Traduction en Python

```

1 for i in range(11):
2     print("Bonjour")
    
```

Attention !

La fonction *range* exclut la dernière valeur

[boucle while](#)

La boucle *while* permet de répéter une instruction ou un bloc d'instructions qu'une certaine condition reste vraie.

Exemple en algorithmique

```

1 i=0
2 tant que i <11 faire:
3     Ecrire("Bonjour")
4     i<-i+1
    
```

Traduction en Python

```

1 i=0
2 while i <11:
3     print("Bonjour")
4     i=i+1
    
```

Attention !

La fonction *while* peut produire une boucle infinie si la condition est toujours valide !

Remarque

Généralement, la boucle *for* s'utilise lorsque le nombre de répétition des instructions est connu avant le lancement du programme.

Par contre la boucle *while* s'utilise lorsqu'on répète en fonction d'une condition (On ne sait pas le nombre exacte de répétitions).

Les ruptures de boucles

break pour la boucle *for* et *while*

Le mot *break* utilisé dans les boucles *for* permet d'arrêter la boucle :

Le mot break

```

1 for i in range(10):
2     print(i)          #sera répétée 10 fois
3 print("Fin du programme")

1 for i in range(10):
2     print(i)          # Va afficher la 1ere valeur de i: 0
3     break            # Arrête la boucle for
4 print("Fin du programme")
    
```

le mot clé *continue* pour la boucle *while*

Le mot clé *continue* permet de retourner à la fonction répétitive de base, *continue* n'a pas d'effet remarquable sur le fonctionnement de la boucle *for*. Par contre, son utilisation dans la boucle *while* peut présenter une boucle infinie :

Le mot clé *continue*

```

1 i=0
2 while i < 10:
3     print(i)    # Va afficher la valeur de i pour chaque itération 0,1,2...
4     i=i+1
5 print("Fin du programme")
6 #####

1 i=0
2 while i < 10:
3     print(i)    # va afficher la première valeur de i:0
4     continue   # renvoie l'exécution à la fonction répétitive while
5     i=i+1      # cette ligne ne sera jamais atteinte
6 print("Fin du programme")
7 # ce deuxième script produira une boucle infinie!

```

4.2 Les commentaires en Python

Les langages de programmation proposent une notation pour insérer des commentaires dans le code, c'est-à-dire du texte qui va être ignoré par l'ordinateur. Les commentaires peuvent servir à expliquer une partie compliquée d'un programme, ou à mettre des indications dans le code. Pour insérer un commentaire en Python, on précède le texte à écrire par un `#`.

Exemple:

```

1 #ce programme calcule et affiche la somme de deux nombres x et y
2 #ceci est un commentaire
3 #il est ignoré lors de l'exécution
4 x=20    #Le premier nombre
5 y=10    # le deuxième nombre
6 print(x+y) # opération et affichage

```